

Promptathon - ICEPOT Prompt Makeover Challenge

This document contains the completed assignment documentation for the Promptathon challenge. The purpose of this activity is to improve weak AI prompts by using the ICEPOT framework

ICEPOT framework:

- **I — Instruction** → What exactly should AI do?
- **C — Context** → Application/environment/domain details
- **E — Example** → Sample test case/scenario
- **P — Persona** → Who AI should act as (example: Senior QA Engineer)
- **O — Output Format** → Table/Gherkin/list/etc.
- **T — Tone** → Formal/technical/simple audience

Objective

The objective is to compare a simple/weak prompt with an improved structured prompt. We identify missing details, rebuild the prompt using ICEPOT, run both prompts in ChatGPT, and observe the improvement in output quality.

Process Followed

1. Selected 3 scenarios from the given list.
2. Reviewed the weak prompt provided for each scenario.
3. Identified missing information in the weak prompt.
4. Created an improved prompt using the ICEPOT framework.
5. Ran the weak prompt in ChatGPT and saved the output.
6. Ran the improved ICEPOT prompt in ChatGPT and saved the output.
7. Compared both outputs and documented observations.

Scenario 1: API Testing

Weak Prompt:

Write API test cases for user registration.

Missing Information:

- No HTTP method details
- No request and response format
- No validation points
- No expected status codes
- No testing tool context

ICEPOT:

Instruction: Create detailed API test cases for a user registration API covering positive, negative, validation, and security scenarios.

Context: The application is a web-based user management system. Testing is performed using Postman.

Example: A valid registration request should return HTTP 201 Created with a success message.

Persona: Act as a Senior API QA Engineer.

Output Format: Provide a table with Test Case ID, Scenario, Request Data, Expected Result, and Status Code.

Tone: Professional and technical.

Scenario 2: File Upload

Weak Prompt:

Generate test cases for file upload.

Missing Information:

- No supported file types
- No file size limit
- No failure scenarios
- No security validation

ICEPOT:

Instruction: Generate complete test cases for a file upload feature.

Context: The feature belongs to an employee document management portal where users upload PDF, JPG, and PNG files.

Example: Uploading a valid PDF file within the allowed size should succeed.

Persona: Act as a Senior Software Test Engineer.

Output Format: Provide a table with Test ID, Scenario, Test Data, and Expected Result.

Tone: Formal and QA documentation style.

Scenario 3: Role-Based Access Control

Weak Prompt:

Write test cases for user roles.

Missing Information:

- No role details
- No permission rules
- No application context
- No unauthorized access cases

ICEPOT:

Instruction: Create test cases for Role-Based Access Control.

Context: The application is a banking portal with Admin, Manager, and Viewer roles.

Example: Viewer users should not be allowed to modify account settings.

Persona: Act as a Security-focused QA Engineer.

Output Format: Provide a table with Test Case ID, Role, Action, and Expected Access Result.

Tone: Professional and security-focused.

Expected Observations After Comparison

- The ICEPOT prompt provides more context, so ChatGPT gives more accurate and relevant answers.

- The improved prompt produces structured output that is easier for QA teams to use.
- Adding persona, format, and specific requirements improves the quality of AI responses.